

AutoTraQ Cheat Sheet — Dean (Database)

Your Section: Database (MySQL + Prisma Schema)

Quick Overview

AutoTraQ uses **MySQL** as its relational database, managed through **Prisma ORM**. The schema is defined in a single file (`schema.prisma`) which serves as the **single source of truth** — Prisma auto-generates the database tables and a type-safe client from it.

Tech Stack

- **MySQL 8** — relational database (hosted on Railway in production)
 - **Prisma ORM** — schema-first database toolkit
 - **Prisma Migrate** — database migration system
 - **Prisma Client** — auto-generated, type-safe query builder
-

Schema Location

```
backend/prisma/schema.prisma    ← THE file. This defines everything.
```

Data Models (Tables)

Core Models:

Model	Table Name	Purpose
User	<code>users</code>	Team members — email, hashed password, role
Part	<code>parts</code>	Parts catalog — SKU, name, condition, cost, min stock
InventoryEvent	<code>inventory_events</code>	Stock movements — in/out with quantity, reason, user
Vehicle	<code>vehicles</code>	Vehicle records — VIN, year, make, model
Request	<code>requests</code>	Part requests — who needs what, approval status

Supporting Models:

Model	Table Name	Purpose
RoleRequest	role_requests	Users requesting role promotions
PartImage	part_images	Photos attached to parts
VehiclePart	vehicle_parts	Many-to-many: which parts fit which vehicles
Interchange	interchanges	Part compatibility/cross-reference data
PartHierarchy	part_hierarchy	Category tree for organizing parts
SKUCounter	sku_counters	Auto-incrementing SKU generation per category

Key Relationships

```

User —< InventoryEvent  (one user creates many stock events)
User —< Request          (one user creates many requests)
Part —< InventoryEvent   (one part has many stock movements)
Part —< PartImage        (one part has many images)
Part >—< Vehicle         (many-to-many via vehicle_parts)
Part —< Interchange      (one part has many cross-references)
PartHierarchy —< Part    (category tree organizes parts)

```

Enums (Defined in Schema)

```

Role:          admin | manager | fulfillment | viewer
PartCondition: NEW | USED | REFURBISHED | UNKNOWN
RequestStatus: PENDING | APPROVED | DENIED | FULFILLED | CANCELLED
RoleRequestStatus: PENDING | APPROVED | DENIED
EventType:     IN | OUT | ADJUSTMENT | RETURN

```

How Prisma Works

1. Define schema → schema.prisma

```

model Part {
  id          Int          @id @default(autoincrement())
  sku         String       @unique
  name        String
  condition   PartCondition @default(UNKNOWN)
  minStock    Int          @default(5)

```

```
  @@map("parts")    // ← maps to MySQL table name
}
```

2. Push to database → creates/updates MySQL tables

```
npx prisma db push
```

3. Generate client → creates type-safe query builder

```
npx prisma generate
```

4. Use in code (services layer)

```
// Find all low-stock parts
const lowStock = await prisma.part.findMany({
  where: { onHand: { lt: prisma.part.fields.minStock } }
});

// Create a new inventory event
await prisma.inventoryEvent.create({
  data: { partId: 1, type: 'IN', quantity: 50, userId: user.id }
});
```

Seed Scripts (Pre-populated Data)

```
npm run db:seed-admins    # Creates team admin accounts
npx prisma db seed        # Seeds parts, vehicles, SKU data
```

Located in: - `backend/src/scripts/seed-admin-accounts.ts` — team accounts -
`backend/prisma/seed-parts.ts` — sample parts catalog - `backend/prisma/seed-vehicles.ts` — sample vehicles - `backend/prisma/seed-sku.ts` — SKU counter initialization

Useful Prisma Commands

```
npx prisma studio          # Visual database browser (opens in browser)
npx prisma db push         # Sync schema → MySQL (dev)
npx prisma migrate dev     # Create migration file (production)
npx prisma generate        # Regenerate client after schema changes
npx prisma format          # Format schema.prisma
```

Talking Points for the Meeting

- "We chose **MySQL** for its reliability and strong relational support — parts inventory naturally has many relationships (parts ↔ vehicles, parts ↔ events, users ↔ requests)"
- "**Prisma ORM** gives us a schema-first approach — the `schema.prisma` file is the single source of truth for the entire database structure"
- "The schema uses **proper normalization** — no data duplication, foreign keys enforce referential integrity"
- "**Enums** enforce valid states at the database level — a part condition can only be NEW, USED, REFURBISHED, or UNKNOWN"
- "The **PartHierarchy** model implements a tree structure for organizing parts into categories and subcategories"
- "**Inventory events** use an append-only log pattern — we never delete stock records, we create IN/OUT events. This gives us a full audit trail of every movement"
- "**Seed scripts** let us bootstrap the database with realistic data for development and demos"
- "Prisma auto-generates a **type-safe client** — if the schema changes, TypeScript catches any broken queries at compile time"